MODUL 4

KOMUNIKASI LONG RANGE (LoRa)

TUJUAN

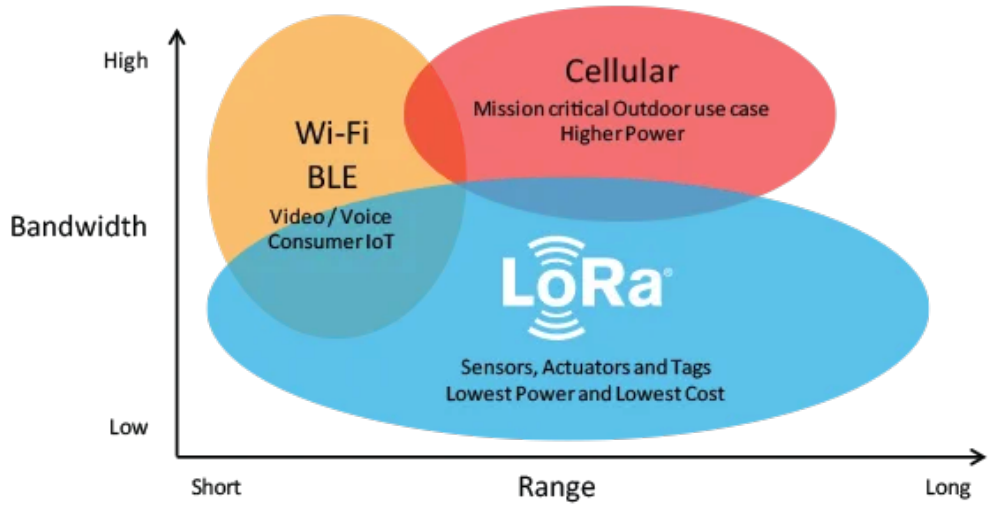
1) Mengetahui teknologi modulasi Long Range (LoRa).
2) Memahami komunikasi menggunakan LoRa, terutama LoRaWAN.

REQUIREMENT

1) Arduino IDE/PlatformIO,
2) LoRa Module/Cosmic LoRa Aurora V2 ESP32 LoRa Module,
3) Akun ANTARES,
4) Sensor (opsional),
5) Actuator (opsional)

4.1 LoRa

LoRa (Long Range) adalah teknik modulasi radio spread spectrum untuk komunikasi jarak jauh dengan laju data rendah dan konsumsi daya yang relatif rendah. LoRa bekerja pada pita frekuensi sub-GHz yang ketentuannya berbeda menurut wilayah. Teknologi ini sesuai untuk pengiriman paket data kecil pada aplikasi seperti pertanian pintar, kota pintar, pemantauan industri, dan penginderaan lingkungan. LoRa bukan protokol jaringan; protokol jaringan yang umum dibangun di atas LoRaWAN.



Gambar 4.1: LoRa Bandwidth - Range

LoRa menggunakan Chirp Spread Spectrum (CSS) dan teknik koreksi kesalahan untuk meningkatkan ketahanan penerimaan sinyal. Parameter seperti spreading factor, bandwidth, coding rate, frekuensi, dan daya pancar memengaruhi jangkauan, airtime, ketahanan gangguan, serta laju data. Topologi star-of-stars merupakan karakteristik arsitektur LoRaWAN, bukan sifat wajib komunikasi LoRa peer-to-peer.

4.2 Alur Komunikasi LoRa

Komunikasi LoRa menggunakan pita frekuensi yang mengikuti ketentuan wilayah dan regulasi setempat. Contohnya, LoRaWAN menggunakan rencana regional EU863-870 di sebagian Eropa dan US902-928 di Amerika Serikat. Untuk Indonesia, parameter regional yang digunakan adalah AS923-2 pada pita **920–923 MHz**. Pengguna harus memastikan frekuensi, daya pancar, duty cycle, dan sertifikasi perangkat sesuai regulasi yang berlaku.

Pilihan frekuensi memengaruhi jangkauan dan kinerja komunikasi. Perangkat akhir, seperti sensor IoT atau objek yang terhubung, menggunakan modulasi LoRa untuk mengubah data yang dikodekan menjadi sinyal chirp, yang ditransmisikan melalui gelombang udara. LoRa dapat mengirim data dengan menggunakan teknik modulasi yang memungkinkan transmisi informasi secara efisien dalam jarak jauh sekaligus menghemat daya.

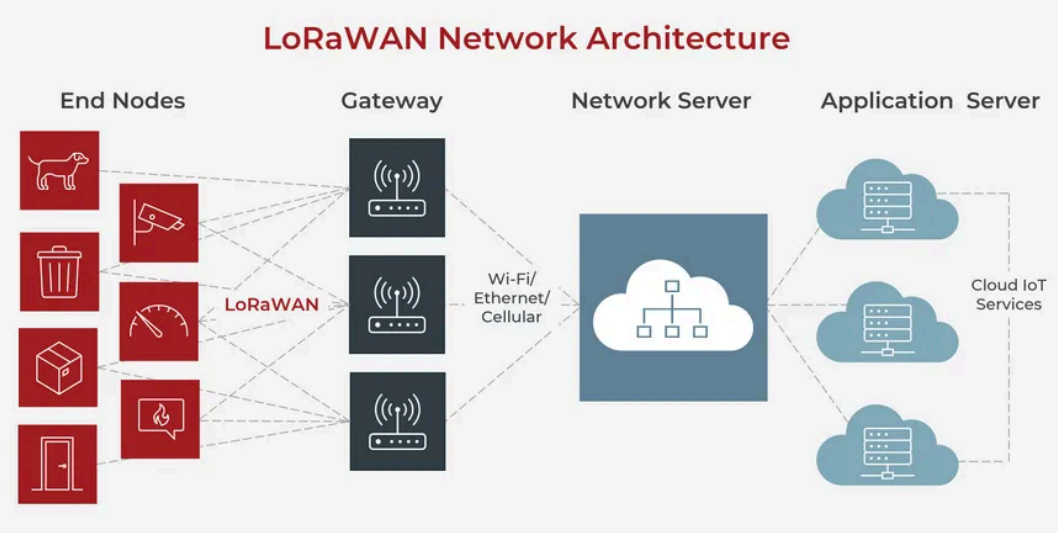
LoRa menggunakan modulasi **Chirp Spread Spectrum (CSS)**, yaitu sinyal chirp yang frekuensinya berubah terhadap waktu. Pada komunikasi peer-to-peer, transmitter dan receiver berkomunikasi langsung apabila menggunakan parameter radio yang kompatibel, seperti frekuensi, bandwidth, spreading factor, coding rate, dan sync word. Pada jaringan LoRaWAN, paket dari end-device dapat diterima oleh satu atau beberapa gateway, kemudian diteruskan melalui jaringan IP ke network server. Dengan demikian, keberadaan gateway dan server merupakan bagian dari LoRaWAN, bukan persyaratan untuk seluruh komunikasi LoRa.

4.3 LoRaWAN

LoRaWAN (Long Range Wide Area Network) merupakan protokol komunikasi jaringan yang dibangun di atas teknologi modulasi LoRa. Jika LoRa mengatur bagaimana sinyal radio dikirimkan melalui udara, LoRaWAN mengatur bagaimana perangkat bergabung ke dalam jaringan, mengirim dan menerima data, melakukan autentikasi, menjaga keamanan data, serta meneruskan data dari perangkat menuju aplikasi pengguna.

Secara umum, LoRaWAN melibatkan empat komponen utama, yaitu:

a. **End-device**, berupa sensor, aktuator, atau perangkat IoT yang mengumpulkan dan mengirimkan data.
b. **Gateway**, yang menerima paket LoRa dari berbagai **end-device** dan meneruskannya melalui jaringan IP.
c. **Network server**, yang mengelola perangkat, memeriksa keamanan paket, menghilangkan data duplikat, dan menentukan jalur pengiriman data.
d. **Application server**, yang mengolah dan menampilkan data agar dapat digunakan oleh pengguna atau sistem aplikasi.



Gambar 4.2: Arsitektur LoRaWAN

LoRaWAN menggunakan topologi **star-of-stars**. Pada topologi ini, **end-device** tidak harus terhubung dengan satu gateway tertentu. Satu paket data dapat diterima oleh beberapa gateway sekaligus, kemudian diteruskan ke **network server**. Server akan memilih dan memproses paket yang sesuai serta menghapus paket duplikat. Pendekatan ini dapat meningkatkan jangkauan dan keandalan komunikasi tanpa mengharuskan perangkat akhir melakukan proses **routing** seperti pada jaringan **mesh**.

Penggunaan LoRaWAN sesuai untuk aplikasi yang hanya mengirimkan data berukuran kecil secara berkala atau berdasarkan suatu kejadian, seperti data suhu, kelembapan, kualitas udara, posisi aset, ketinggian air, status meter, kondisi mesin, dan status perangkat. LoRaWAN dipilih karena mampu menyediakan jangkauan komunikasi yang luas, konsumsi daya rendah, kapasitas perangkat yang besar, serta biaya pembangunan jaringan yang relatif rendah.

Alur komunikasi LoRaWAN dimulai ketika **end-device** membaca data dari sensor, kemudian mengemas dan mengenkripsi data tersebut sesuai format LoRaWAN. Data dikirimkan menggunakan modulasi LoRa dan dapat diterima oleh satu atau beberapa gateway. Gateway meneruskan paket melalui koneksi internet atau jaringan lokal menuju **network server**. Setelah dilakukan autentikasi, pemeriksaan integritas, penghapusan paket duplikat, dan pengelolaan jaringan, data diteruskan ke **application**

server untuk disimpan, dianalisis, atau ditampilkan pada dashboard. Untuk komunikasi **downlink**, data dikirimkan kembali dari **application server** menuju **end-device** melalui **network server** dan gateway.

LoRaWAN sebaiknya digunakan ketika suatu sistem memiliki karakteristik berikut:

a. Membutuhkan jangkauan komunikasi hingga beberapa kilometer;
b. Menggunakan perangkat bertenaga baterai yang harus bertahan lama;
c. Memiliki banyak sensor yang tersebar di wilayah luas;
d. Hanya mengirimkan data dalam ukuran kecil dengan interval tertentu;
e. Tidak membutuhkan kecepatan data tinggi atau komunikasi waktu nyata;
f. Membutuhkan keamanan, autentikasi perangkat, dan pengelolaan jaringan secara terpusat.

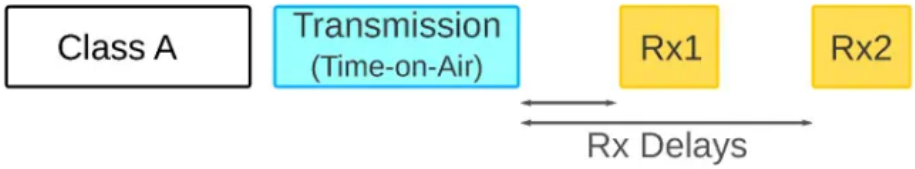
Sebaliknya, LoRaWAN kurang sesuai untuk aplikasi yang membutuhkan pengiriman video, audio, file berukuran besar, kontrol dengan latensi sangat rendah, atau komunikasi data secara terus-menerus. Untuk kebutuhan tersebut, teknologi seperti Wi-Fi, Ethernet, jaringan seluler, atau protokol komunikasi lain dengan kapasitas data lebih tinggi lebih sesuai.

4.4 Tipe Kelas LoRaWAN

Kelas perangkat LoRaWAN terdiri atas Kelas A, Kelas B, dan Kelas C. Kelas tersebut mengatur waktu perangkat membuka jendela penerimaan downlink dan berdampak pada latensi, konsumsi daya, serta pola komunikasi. Semua end-device LoRaWAN wajib mendukung Kelas A; dukungan Kelas B dan Kelas C bersifat tambahan.

4.4.1 Class A

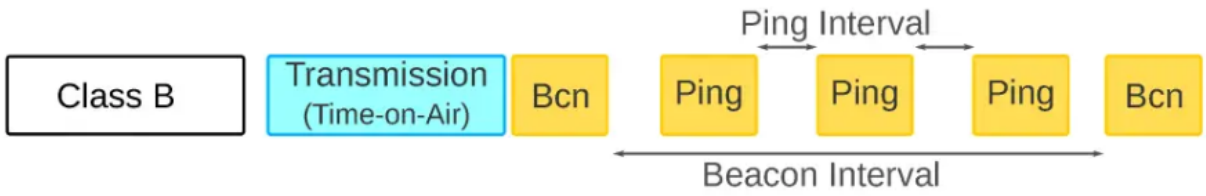
Pada Kelas A, komunikasi uplink dimulai oleh end-device. Setelah setiap uplink, perangkat membuka dua jendela penerimaan downlink. Pola ini memberikan konsumsi daya paling rendah, tetapi downlink harus menunggu hingga perangkat melakukan uplink.



Gambar 4.3: Komunikasi LoRa Class A

4.4.2 Class B

Kelas B menambahkan beacon jaringan dan ping slot terjadwal di samping mekanisme Kelas A. Dengan jadwal tersebut, server memiliki peluang downlink yang lebih dapat diprediksi, tetapi konsumsi daya dan kompleksitas sinkronisasi lebih tinggi daripada Kelas A.



Gambar 4.4: Komunikasi LoRa Class B

4.4.3 Class C

Kelas C menjaga penerima tetap terbuka hampir sepanjang waktu, kecuali ketika perangkat sedang mengirim. Latensi downlink menjadi paling rendah, tetapi konsumsi daya paling tinggi sehingga umumnya digunakan pada perangkat dengan sumber daya yang stabil.



Gambar 4.5: Komunikasi LoRa Class C

4.5 ANTARES

ANTARES adalah platform horizontal Internet of Things yang didevelop oleh PT Telkom Indonesia TBK. Platform ini menyediakan layanan untuk mengatur dan mengkoneksikan perangkat IoT, menyimpan data sensor, dan menyajikan data melalui dashboard ataupun aplikasi lainnya. ANTARES melayani berbagai implementasi IoT, termasuk smart homes, smart metering, tracking asset, dan smart buildings.

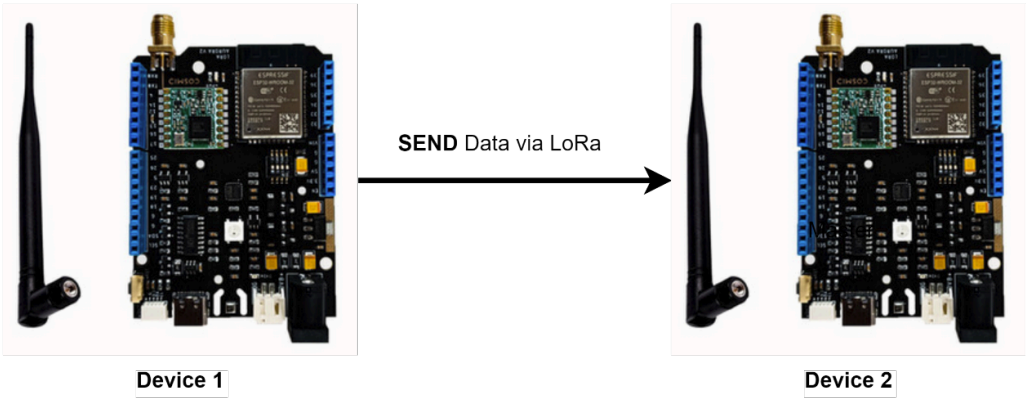


Gambar 4.6: ANTARES Logo

ANTARES memiliki dua layanan utama: IoT platform dan IoT connectivity. Layanan connectivity melayani teknologi seperti LoRaWAN, NB-IoT, dan GPRS, memperbolehkan perangkat IoT untuk mentransmit data ke platform ANTARES untuk monitoring dan pemrosesan. ANTARES merupakan IoT Platform yang akan kita pakai di implementasi pada bagian berikutnya.

4.6 Implementasi Protokol LoRa

4.6.1 Peer-to-peer LoRa



Gambar 4.7: Visualisasi P2P LoRa

Implementasi peer-to-peer ini menggunakan modulasi LoRa dengan alur seperti pada gambar di atas.

4.6.1.1 Cosmic LoRa Aurora V2

Cosmic LoRa Aurora V2 merupakan dev board LoRa yang menggunakan transceiver LoRa RFM95W yang dikeluarkan oleh Cosmic dan dapat dilihat pada Lampiran 4. Board ini memakai ESP32 sebagai mikrokontrolernya sehingga selain memiliki LoRa, board ini juga sudah dapat menggunakan konektivitas Wi-Fi. Selain itu, LoRa Aurora V2 ini memiliki 3 sumber tenaga yang dapat digunakan, yaitu USB, LiPo Battery, dan DC Jack dan sudah memiliki sistem smart power switching sehingga secara otomatis dapat memilih sumber tenaga yang digunakan, seperti jika LiPo Battery sedang diisi, maka sistem dapat secara otomatis berganti ke USB atau DC Jack.

4.6.1.2 LoRa Library by Sandeep Mistry

Library LoRa yang dikembangkan oleh Sandeep Mistry ini adalah library yang akan digunakan pada implementasi LoRa menggunakan Cosmic LoRa Aurora V2. Library ini merupakan library yang digunakan untuk mengirim dan menerima data menggunakan komunikasi LoRa dengan header `LoRa.h`.

Information
Library yang digunakan berasal dari repositori berikut:
https://github.com/sandeepmistry/arduino-lora

4.6.1.3 Source Code LoRa Peer-to-Peer

Pada kode sumber LoRa peer-to-peer baik untuk transmitter dan receiver menggunakan contoh kode dari Cosmic yang tersedia pada laman web berikut:

Source Code
https://github.com/cosmic-id/cosmic-lora-aurora/blob/main/examples/aurora-v2-lora-p2p-receiver/aurora-v2-lora-p2p-receiver.ino

- **Transmitter**

a. **Inisialisasi**

Inisialisasi Transmitter LoRa

```
#include <SPI.h>
#include <LoRa.h>

// Tentukan pin yang digunakan oleh modul LoRa
#define LORA_NSS  15
#define LORA_RST  0
#define LORA_DIO0 27
#define LORA_EN   32

#define LORA_TX_POWER         20
#define LORA_SPREADING_FACTOR 12

int counter = 0;
```

Dalam menginisialisasi P2P LoRa, kita akan menggunakan library `SPI.h` dan `LoRa.h` oleh Sandeep Mistry. Setelah itu tentukan pin-pin LoRa mana yang digunakan di setiap `#define`. Pada kode di atas ada variabel tambahan yang diinisialisasi, yaitu variabel `counter` yang akan digunakan sebagai nilai yang akan dikirim ke receiver.

b. **Setup**

Setup Transmitter LoRa

```
void setup() {
  pinMode(LORA_EN, OUTPUT);     // Inisialisasi mode pin LoRa
  digitalWrite(LORA_EN, HIGH);  // LoRa chip dinyalakan

  // Inisialisasi Serial Monitor
  Serial.begin(115200);
  while (!Serial);
  Serial.println("LoRa Sender");

  // Setup pin modul LoRa
  LoRa.setPins(LORA_NSS, LORA_RST, LORA_DIO0);

  // Ganti LoRa.begin(---E-) argument dengan frekuensi yang sesuai regulasi wilayah
```

```
  // 433E6 untuk region Asia
  // 866E6 untuk region Europe
  // 915E6 untuk region North America
  // 920E6 untuk Indonesia
  while (!LoRa.begin(920E6)) {
    Serial.println(".");
    delay(500);
  }

  LoRa.setTxPower(LORA_TX_POWER, PA_OUTPUT_PA_BOOST_PIN);
  LoRa.setSpreadingFactor(LORA_SPREADING_FACTOR);

  // Ganti sync word (0xF3) menyesuaikan transceiver
  // Sync word membantu menyaring paket dengan nilai berbeda
  // Memiliki rentang 0x00–0xFF
  LoRa.setSyncWord(0xF3);
  Serial.println("LoRa Initializing OK!");
}
```

Pada bagian `setup()` untuk LoRa sebagai transmitter, ada beberapa hal yang harus kita tentukan:

- **Deklarasi pinMode** untuk pin Enable (Berdasarkan kode, maka pin enable terdapat pada konstanta `LORA_EN` dan mengubah state-nya menjadi `HIGH`.
- **Setup pin** untuk modul LoRa menggunakan fungsi `LoRa.setPins(NSS_PIN, RST_PIN, DI00_PIN)`.
- **Memulai modul LoRa** menggunakan fungsi `LoRa.begin(frequency)` dengan parameter `frequency` diisi dengan frekuensi yang sesuai regulasi wilayah. Untuk contoh praktikum di Indonesia digunakan `920E6` pada pita 920–923 MHz (AS923-2). Pastikan frekuensi dan daya pancar sesuai regulasi serta spesifikasi perangkat.
- **Menentukan sync word** yang disesuaikan antara transmitter dan receiver. Sync word merupakan parameter sinkronisasi, bukan alamat perangkat atau mekanisme keamanan. Nilainya berada pada rentang 0x00–0xFF dan ditentukan menggunakan fungsi `LoRa.setSyncWord(sync_word)`.

c. **Loop**

Loop Transmitter LoRa

```
void loop() {
  Serial.print("Sending packet: ");
  Serial.println(counter);

  //Send LoRa packet to receiver
  LoRa.beginPacket();
  LoRa.print("hello ");
  LoRa.print(counter);
  LoRa.endPacket();

  counter++;
  delay(5000);
}
```

- **Receiver**

a. **Inisialisasi**

Inisialisasi Receiver LoRa

```
#include <SPI.h>
#include <LoRa.h>

// Tentukan pin yang digunakan oleh modul LoRa
```

```
#define LORA_NSS  15
#define LORA_RST  0
#define LORA_DIO0 27
#define LORA_EN   32

#define LORA_TX_POWER         20
#define LORA_SPREADING_FACTOR 12

```

Tidak ada perbedaan antara inisialisasi untuk transmitter dan receiver, seperti pin dan library yang digunakan. Namun, pada kode di atas tidak ada tambahan variabel `counter` seperti pada transmitter yang digunakan sebagai data yang akan dikirim melalui LoRa.

b. **Setup**

Setup Receiver LoRa

```
void setup() {
  pinMode(LORA_EN, OUTPUT);     // Inisialisasi mode pin LoRa
  digitalWrite(LORA_EN, HIGH);  // LoRa chip dinyalakan

  // Inisialisasi Serial Monitor
  Serial.begin(115200);
  while (!Serial);
  Serial.println("LoRa Receiver");

  // Setup pin modul LoRa
  LoRa.setPins(LORA_NSS, LORA_RST, LORA_DIO0);

  // Ganti LoRa.begin(---E-) argument dengan frekuensi yang sesuai regulasi wilayah
  // 433E6 untuk region Asia
  // 866E6 untuk region Europe
  // 915E6 untuk region North America
  // 920E6 untuk Indonesia
  while (!LoRa.begin(920E6)) {
    Serial.println(".");
    delay(500);
  }

  LoRa.setTxPower(LORA_TX_POWER, PA_OUTPUT_PA_BOOST_PIN);
  LoRa.setSpreadingFactor(LORA_SPREADING_FACTOR);

  // Ganti sync word (0xF3) menyesuaikan transceiver
  // Sync word membantu menyaring paket dengan nilai berbeda
  // Memiliki rentang 0x00–0xFF
  LoRa.setSyncWord(0xF3);
  Serial.println("LoRa Initializing OK!");
}
```

Pada bagian `setup()` di sisi receiver, perlu diperhatikan pada bagian frekuensi dan sync word yang digunakan. Kedua hal tersebut harus memiliki nilai yang sama baik dari sisi pengirim (transmitter) dan penerima (receiver). Pada kode sumber kali ini, frekuensi yang digunakan adalah 920E6 dan untuk sync word menggunakan nilai 0xF3.

c. **Loop**

Loop Receiver LoRa

```
void loop() {
  // Mencoba untuk parse paket yang diterima
  int packetSize = LoRa.parsePacket();

  if (packetSize) {
    // Menerima paket
```

```
    Serial.print("Received packet '");

    // Membaca paket yang diterima
    while (LoRa.available()) {
      String LoRaData = LoRa.readString();
      Serial.print(LoRaData);
    }

    // Menampilkan RSSI/Sinyal dari paket yang diterima
    Serial.print("' with RSSI ");
    Serial.println(LoRa.packetRssi());
  }
}
```

Pada bagian `loop()`, receiver akan menerima data dari transceiver yang memiliki sync word yang sama. `LoRa.parsePacket()` akan memeriksa paket yang datang. Jika ada paket yang datang, maka paket akan dibaca menggunakan `LoRa.readString()` selama paket tersedia sesuai dengan kondisi `LoRa.available()`. Pada bagian terakhir, `LoRa.packetRssi()` akan mendapatkan RSSI atau kekuatan sinyal dari paket LoRa yang didapat dalam satuan dBm.

4.6.2 Implementasi LoRaWAN Menggunakan ANTARES

Proses komunikasi dapat dijelaskan sebagai berikut:

End Device → LoRaWAN Gateway → Platform ANTARES

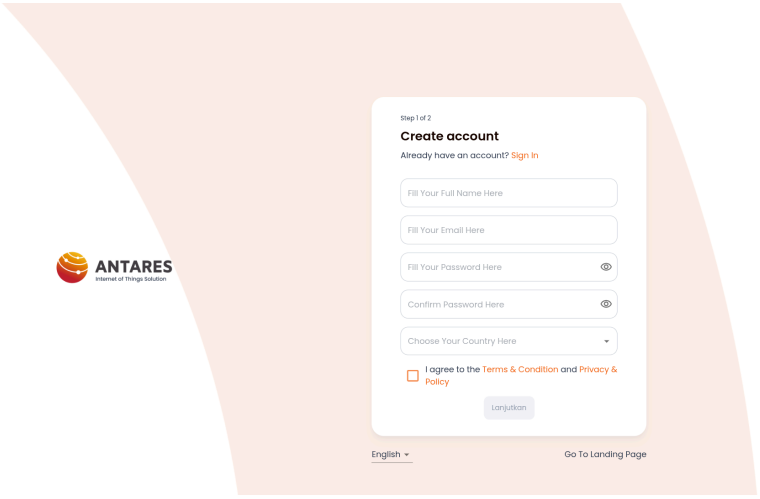
End device mengirimkan data menggunakan modulasi LoRa dan protokol LoRaWAN. Gateway menerima paket radio dan meneruskannya ke infrastruktur ANTARES melalui koneksi Internet.

Implementasi ini menggunakan LoRaWAN Class A dan Activation by Personalization atau ABP. Pada aktivasi ABP, perangkat menggunakan alamat perangkat dan kunci sesi yang telah ditentukan sebelumnya tanpa melakukan prosedur join setiap kali perangkat dinyalakan.

4.6.2.1 Membuat Application dan Device di ANTARES

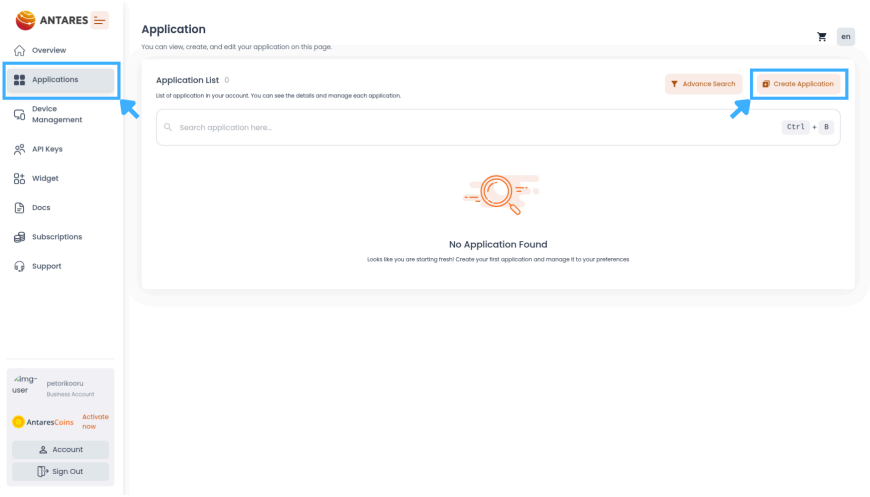
Sebelum mengunggah program, buat sebuah application dan device pada konsol ANTARES. Application dan device dibuat melalui langkah-langkah berikut:

a. Masuk ke konsol ANTARES.



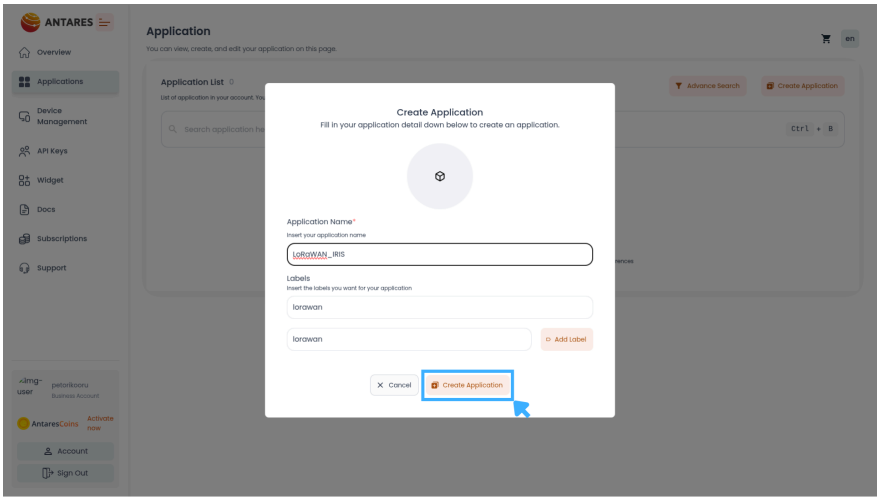
Gambar 4.8: Pendaftaran ANTARES

b. Buka halaman **Application**.



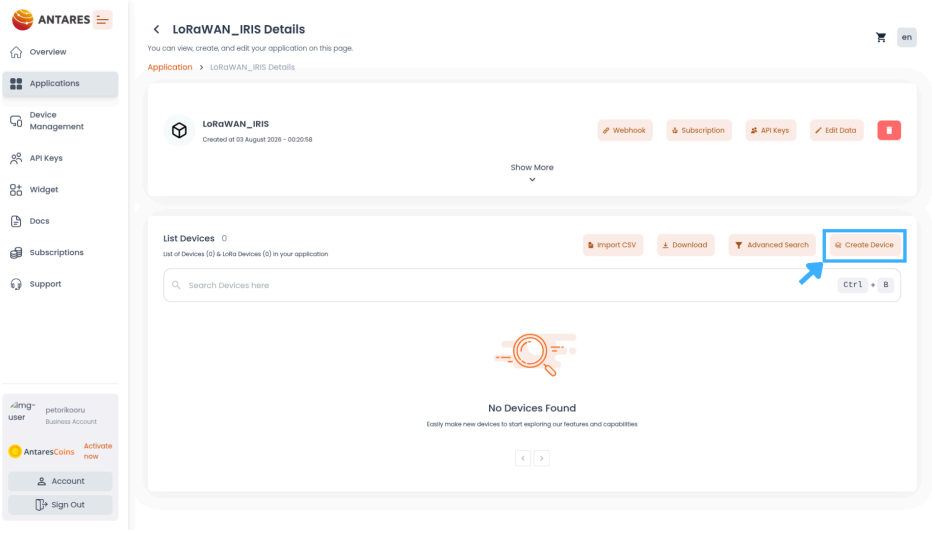
Gambar 4.9: Dashboard Application ANTARES

c. Buat application baru dengan tombol **Create Application**.

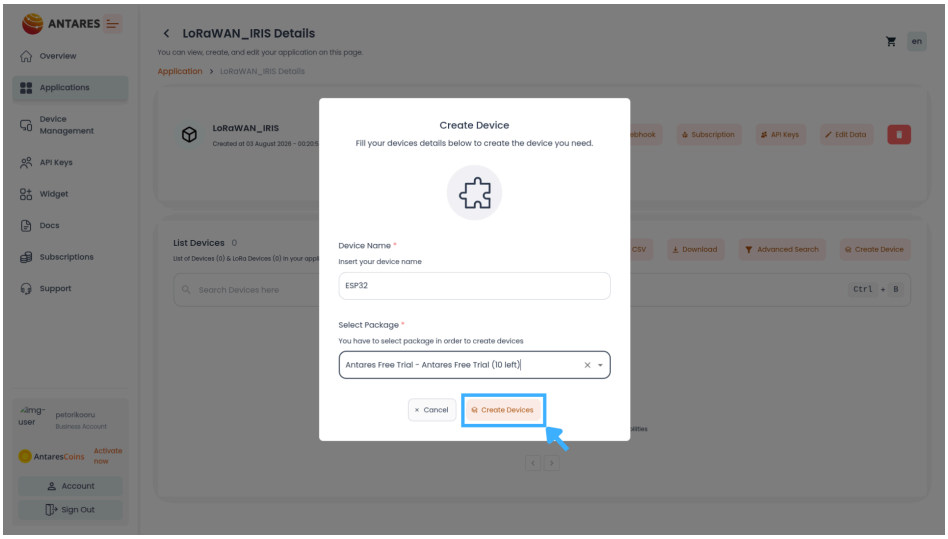


Gambar 4.10: Dialog Pembuatan Application ANTARES

d. Selanjutnya, halaman akan dialihkan ke halaman device. Buat device baru dengan tombol **Create Device**.

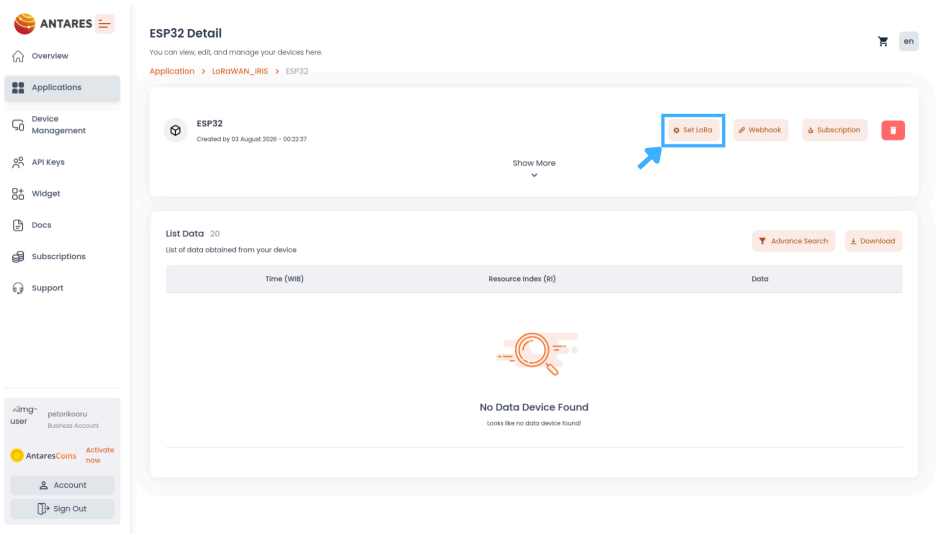


Gambar 4.11: Halaman Device pada Application ANTARES



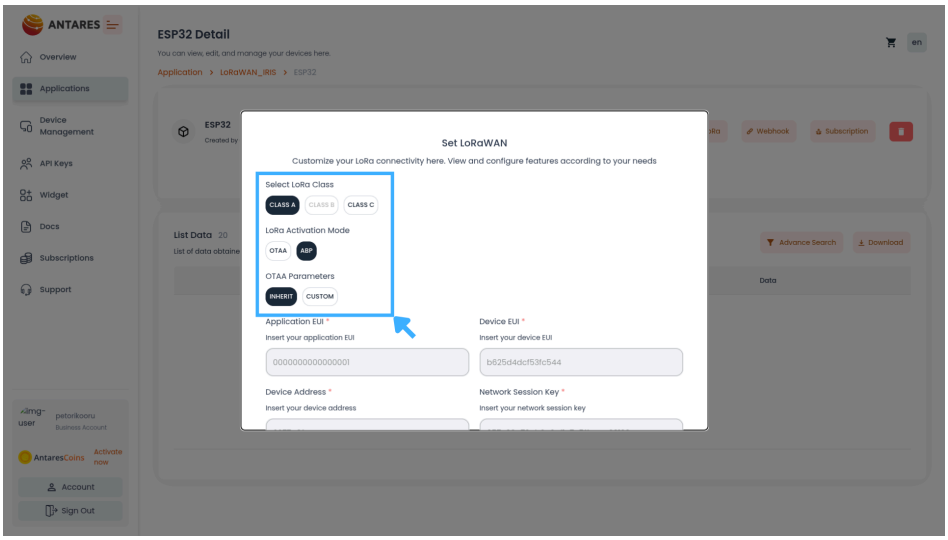
Gambar 4.12: Dialog Pembuatan Device ANTARES

e. Setelah itu, halaman akan dialihkan ke detail device. Pilih **Set LoRa**.



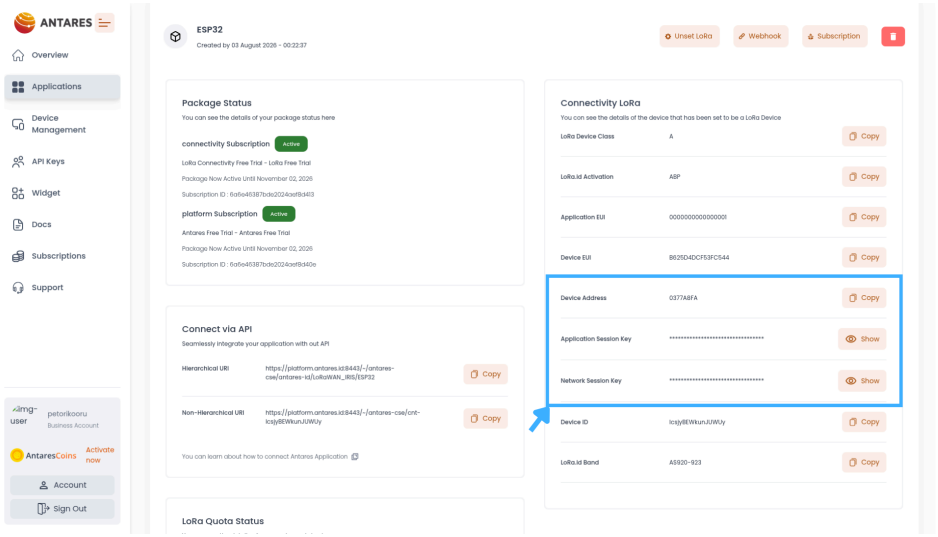
Gambar 4.13: Detail Device ANTARES

f. Konfigurasikan pengaturan LoRa seperti berikut:



Gambar 4.14: Detail Device ANTARES

g. Terakhir, simpan parameter LoRaWAN yang dihasilkan. Parameter yang diperlukan dapat dilihat pada gambar berikut.



Gambar 4.15: Parameter LoRaWAN Device ANTARES

ANTARES menghasilkan parameter ABP berikut:

- **Device Address** atau `DevAddr`;
- **Network Session Key** atau `NwkSKey`;
- **Application Session Key** atau `AppSKey`.

Salin dan simpan parameter tersebut sebelum menyelesaikan konfigurasi device. Parameter tersebut harus dimasukkan ke dalam program Arduino.

Information
Network Session Key dan Application Session Key merupakan identitas rahasia. Jangan memublikasikan kunci yang sebenarnya dalam laporan ataupun repositori publik.

4.6.2.2 Memasang Library LoRaWAN ANTARES

Pasang library LoRaWAN ANTARES bernama `AntaresLoraID` pada Arduino IDE.

Information
Library tersedia melalui dokumentasi berikut:
https://docs.antares.id/pendahuluan/instalasi-library-arduino/antares-lorawan

Program menggunakan file header `lorawan.h` yang disediakan oleh library LoRaWAN ANTARES.

4.6.2.3 Program Source Code LoRaWAN

Program berikut mengonfigurasi Cosmic LoRa Aurora V2 sebagai end device LoRaWAN Class A menggunakan Activation by Personalization atau ABP. Perangkat mengirimkan data counter ke ANTARES setiap 60 detik dan memeriksa pesan downlink setelah setiap transmisi uplink.

Program diawali dengan menyertakan library SPI dan LoRaWAN ANTARES:

Library yang Diperlukan

```
1  #include <SPI.h>
2  #include <lorawan.h>
```

Library `SPI.h` mengelola komunikasi antara ESP32 dan modul radio RFM95W. Library `lorawan.h` menyediakan fungsi-fungsi yang diperlukan untuk mengonfigurasi dan mengoperasikan koneksi LoRaWAN.

Definisi berikut menentukan konfigurasi pin RFM95W yang digunakan oleh Cosmic LoRa Aurora V2:

Konfigurasi Pin Radio LoRa

```
#define LORA_NSS  15
#define LORA_RST  0
#define LORA_DIO0 27
#define LORA_DIO1 2
#define LORA_EN   32

#define SPI_MISO 23
#define SPI_MOSI 19
#define SPI_SCK 18
```

GPIO 15 digunakan sebagai pin chip-select atau NSS. GPIO 0 terhubung ke pin reset, sedangkan GPIO 27 dan GPIO 2 terhubung ke DIO0 dan DIO1. GPIO 32 mengendalikan rangkaian pengaktif radio dan harus diatur menjadi `HIGH` sebelum radio diinisialisasi.

Kredensial ABP yang dihasilkan oleh ANTARES disimpan dalam variabel berikut:

Parameter ABP ANTARES

```
const char *devAddr = "YOUR-DEVICE-ADDRESS";
const char *nwkSKey = "YOUR-NETWORK-SESSION-KEY";
const char *appSKey = "YOUR-APPLICATION-SESSION-KEY";
```

Variabel `devAddr` berisi alamat perangkat LoRaWAN. Variabel `nwkSKey` berisi Network Session Key, sedangkan `appSKey` berisi Application Session Key. Ganti nilai placeholder dengan parameter yang dihasilkan melalui konsol ANTARES.

Variabel berikut menentukan interval transmisi dan menyimpan status program:

Variabel Program

```
const unsigned long interval = 60000;

unsigned long previousMillis = 0;
unsigned int messageCounter = 0;

char message[50];
char receivedData[255];

byte receiveStatus = 0;
int channel = 0;
```

Nilai `interval` dinyatakan dalam milidetik. Nilai `60000` menyebabkan perangkat mengirimkan satu uplink setiap 60 detik. Variabel `previousMillis` menyimpan waktu transmisi sebelumnya, sedangkan `messageCounter` mencatat jumlah pesan yang dihasilkan oleh perangkat. Array `message` menyimpan pesan uplink. Array `receivedData` menyimpan data downlink yang diterima dari ANTARES.

Pemetaan pin RFM95W kemudian diteruskan ke library LoRaWAN:

Pemetaan Pin RFM95W

```
const sRFM_pins RFM_pins = {
  .CS = LORA_NSS,
  .RST = LORA_RST,
```

```
    .DIO0 = LORA_DIO0,
    .DIO1 = LORA_DIO1,
};
```

Struktur `RFM_pins` memungkinkan library berkomunikasi dengan radio RFM95W menggunakan pin ESP32 yang sesuai.

Fungsi `setup()` menginisialisasi Serial Monitor, rangkaian pengaktif radio, antarmuka SPI, modul RFM95W, dan parameter LoRaWAN:

Fungsi Setup

```
void setup() {
  Serial.begin(115200);
  delay(2000);

  pinMode(LORA_EN, OUTPUT);
  digitalWrite(LORA_EN, HIGH);
  delay(100);

  SPI.begin(SPI_SCK, SPI_MISO, SPI_MOSI, LORA_NSS);

  if (!lora.init()) {
    Serial.println("RFM95W radio was not detected");
    while (true) {
      delay(1000);
    }
  }

  Serial.println("RFM95W radio initialized");

  // A Class A device opens two receive windows after each uplink transmission.
  // Therefore, downlink messages are normally delivered after the device sends an uplink.
  lora.setDeviceClass(CLASS_A);

  // The data rate is configured using spreading factor 10 and a bandwidth of 125 kHz:
  // A higher spreading factor generally increases communication range and
  // receiver sensitivity but also increases airtime and reduces the data rate.
  lora.setDataRate(SF10BW125);

  // It will be available to multiple LoRaWAN channels
  lora.setChannel(MULTI);

  // The transmission power should comply with the applicable regional frequency
  // regulations and the requirements of the LoRaWAN network
  lora.setTxPower1(15);

  // ABP credentials
  // Because ABP uses predefined session parameters,
  // the device does not perform an OTAA join procedure before transmitting data.
  lora.setNwkSKey(nwkSKey);
  lora.setAppSKey(appSKey);
  lora.setDevAddr(devAddr);

  Serial.println("LoRaWAN Class A ABP configuration completed");
}
```

Fungsi `loop()` mengendalikan transmisi uplink berkala dan penerimaan downlink:

Loop Utama

```
void loop() {
  unsigned long currentMillis = millis();

  if (currentMillis - previousMillis >= interval) {
```

```
    previousMillis = currentMillis;

    snprintf(message, sizeof(message), "LoRa Counter-%u", messageCounter);

    Serial.print("Sending: ");
    Serial.println(message);

    lora.sendUplink(message, strlen(message), 0, 5);

    messageCounter++;

    channel = lora.getChannel();

    Serial.print("Channel: ");
    Serial.println(channel);
  }

  lora.update();

  receiveStatus = lora.readData(receivedData);

  if (receiveStatus > 0) {
    bool isText = true;

    for (int i = 0; i < receiveStatus; i++) {
      if (!(
        (receivedData[i] >= 32 && receivedData[i] <= 126) ||
         receivedData[i] == 10 || receivedData[i] == 13)
        ){
        isText = false;
        break;
      }
    }

    if (isText) {
      Serial.print("Received text: ");

      for (int i = 0; i < receiveStatus; i++) {
        Serial.print(receivedData[i]);
      }
    } else {
      Serial.print("Received hexadecimal data: ");

      for (int i = 0; i < receiveStatus; i++) {
        if (static_cast<byte>( receivedData[i]) < 0x10) {
          Serial.print('0');
        }

        Serial.print(static_cast<byte>( receivedData[i]), HEX);
        Serial.print(' ');
      }
    }

    Serial.println();
  }
}
```

Pesan uplink dibuat menggunakan nilai message counter saat ini:

Membuat Pesan Uplink

```
snprintf(message, sizeof(message), "LoRa Counter-%u", messageCounter);
```

Sebagai contoh, tiga pesan pertama yang dihasilkan adalah:

Pesan yang Dihasilkan

```
LoRa Counter-0
LoRa Counter-1
LoRa Counter-2
```

Pesan dikirimkan menggunakan fungsi `sendUplink()`:

Mengirim Data ke ANTARES

```
lora.sendUplink(message, strlen(message), 0, 5);
```

Parameter pertama berisi buffer pesan. Parameter kedua berisi jumlah byte yang akan dikirimkan. Parameter ketiga menentukan apakah uplink bersifat confirmed. Nilai `0` memilih unconfirmed uplink. Parameter keempat menentukan LoRaWAN application port 5. Setelah mengirimkan pesan, program meningkatkan message counter dan membaca kanal yang digunakan untuk transmisi:

Informasi Counter dan Kanal

```
messageCounter++;

channel = lora.getChannel();

Serial.print("Channel: ");
Serial.println(channel);
```

Program kemudian memeriksa apakah data downlink telah diterima:

Membaca Data Downlink

```
receiveStatus = lora.readData(receivedData);
```

Nilai kembalian menunjukkan jumlah byte yang diterima. Jika nilainya lebih besar dari nol, program memeriksa apakah setiap byte merepresentasikan karakter yang dapat dicetak.

Apabila seluruh byte yang diterima dapat dicetak, downlink ditampilkan sebagai teks:

Menampilkan Downlink Teks

```
Serial.print("Received text: ");

for (int i = 0; i < receiveStatus; i++) {
  Serial.print(receivedData[i]);
}
```

Sebagai contoh, downlink ANTARES yang memuat teks `ON` ditampilkan sebagai berikut:

Keluaran Downlink Teks

```
Received text: ON
```

Jika data yang diterima mengandung byte yang tidak dapat dicetak, program menampilkan setiap byte dalam format heksadesimal:

Menampilkan Downlink Biner

```
Serial.print("Received hexadecimal data: ");

for (int i = 0; i < receiveStatus; i++) {
  if (static_cast<byte>( receivedData[i]) < 0x10) {
    Serial.print('0');
  }
```

```
7
8     Serial.print(static_cast<byte>( receivedData[i]), HEX);
9     Serial.print(' ');
10  }
```

Downlink biner dapat ditampilkan pada Serial Monitor sebagai berikut:

Keluaran Downlink Biner

```
1  Received hexadecimal data: 01 FF A0
```

4.6.2.4 Menerima Downlink

ANTARES dapat mengirimkan pesan downlink ke perangkat LoRaWAN. Karena program menggunakan Class A, downlink dikirimkan melalui receive window yang dibuka setelah uplink. Ketika downlink teks diterima, Serial Monitor menampilkan keluaran seperti berikut:

Keluaran Downlink Teks

```
1  Received text: ON
```

Ketika downlink berisi data biner yang tidak dapat dicetak, program menampilkannya dalam format heksadesimal:

Keluaran Downlink Biner

```
1  Received hexadecimal data: 01 FF A0
```

Jika perangkat tidak dapat mengirimkan data ke ANTARES, periksa Device Address, Network Session Key, Application Session Key, koneksi antena, jangkauan gateway, konfigurasi pin radio, kelas perangkat, metode aktivasi, dan konfigurasi kanal LoRaWAN.

LAMPIRAN MODUL 4

Source Code untuk Transmitter LoRa

Transmitter LoRa

```
// Menggunakan dua library, yaitu SPI.h dan LoRa.h oleh Sandeep Mistry
#include <SPI.h>
#include <LoRa.h>

// Tentukan pin yang digunakan oleh modul LoRa
#define LORA_NSS  15
#define LORA_RST  0
#define LORA_DIO0 27
#define LORA_EN   32

#define LORA_TX_POWER         20
#define LORA_SPREADING_FACTOR 12

int counter = 0;

void setup() {
  pinMode(LORA_EN, OUTPUT);     // Inisialisasi mode pin LoRa
  digitalWrite(LORA_EN, HIGH);  // LoRa chip dinyalakan

  // Inisialisasi Serial Monitor
  Serial.begin(115200);
  while (!Serial);
  Serial.println("LoRa Sender");

  // Setup pin modul LoRa
  LoRa.setPins(LORA_NSS, LORA_RST, LORA_DIO0);

  // Ganti LoRa.begin(---E-) argument dengan frekuensi yang sesuai regulasi wilayah
  // 433E6 untuk region Asia
  // 866E6 untuk region Europe
  // 915E6 untuk region North America
  // 920E6 untuk Indonesia
  while (!LoRa.begin(920E6)) {
    Serial.println(".");
    delay(500);
  }

  LoRa.setTxPower(LORA_TX_POWER, PA_OUTPUT_PA_BOOST_PIN);
  LoRa.setSpreadingFactor(LORA_SPREADING_FACTOR);

  // Ganti sync word (0xF3) menyesuaikan transceiver
  // Sync word membantu menyaring paket dengan nilai berbeda
  // Memiliki rentang 0x00–0xFF
  LoRa.setSyncWord(0xF3);
  Serial.println("LoRa Initializing OK!");
}

void loop() {
  Serial.print("Sending packet: ");
  Serial.println(counter);

  //Send LoRa packet to receiver
  LoRa.beginPacket();
  LoRa.print("hello ");
  LoRa.print(counter);
  LoRa.endPacket();

  counter++;
  delay(5000);
}
```

Source Code untuk Receiver LoRa

Receiver LoRa

```
// Menggunakan dua library, yaitu SPI.h dan LoRa.h oleh Sandeep Mistry
#include <SPI.h>
#include <LoRa.h>

// Tentukan pin yang digunakan oleh modul LoRa
#define LORA_NSS  15
#define LORA_RST  0
#define LORA_DIO0 27
#define LORA_EN   32

#define LORA_TX_POWER         20
#define LORA_SPREADING_FACTOR 12

void setup() {
  pinMode(LORA_EN, OUTPUT);     // Inisialisasi mode pin LoRa
  digitalWrite(LORA_EN, HIGH);  // LoRa chip dinyalakan

  // Inisialisasi Serial Monitor
  Serial.begin(115200);
  while (!Serial);
  Serial.println("LoRa Receiver");

  // Setup pin modul LoRa
  LoRa.setPins(LORA_NSS, LORA_RST, LORA_DIO0);

  // Ganti LoRa.begin(---E-) argument dengan frekuensi yang sesuai regulasi wilayah
  // 433E6 untuk region Asia
  // 866E6 untuk region Europe
  // 915E6 untuk region North America
  // 920E6 untuk Indonesia
  while (!LoRa.begin(920E6)) {
    Serial.println(".");
    delay(500);
  }

  LoRa.setTxPower(LORA_TX_POWER, PA_OUTPUT_PA_BOOST_PIN);
  LoRa.setSpreadingFactor(LORA_SPREADING_FACTOR);

  // Sync word membantu menyaring paket dengan nilai berbeda
  // Memiliki rentang 0x00–0xFF
  LoRa.setSyncWord(0xF3);
  Serial.println("LoRa Initializing OK!");
}

void loop() {
  // Mencoba untuk membuka paket yang diterima
  int packetSize = LoRa.parsePacket();

  // Menerima paket
  if (packetSize) {
    Serial.print("Received packet '");

    // Membaca paket yang diterima
    while (LoRa.available()) {
      String LoRaData = LoRa.readString();
      Serial.print(LoRaData);
    }

    // Menampilkan RSSI/Sinyal dari paket yang diterima (dB)
    Serial.print("' with RSSI ");
    Serial.println(LoRa.packetRssi());
  }
}
```

LoRaWAN Source Code

LoRaWAN Microcontroller Source Code

```
#include <SPI.h>
#include <lorawan.h>

#define LORA_NSS  15
#define LORA_RST  0
#define LORA_DIO0 27
#define LORA_DIO1 2
#define LORA_EN   32

#define SPI_MISO 23
#define SPI_MOSI 19
#define SPI_SCK 18

const char *devAddr = "YOUR-DEVICE-ADDRESS";
const char *nwkSKey = "YOUR-NETWORK-SESSION-KEY";
const char *appSKey = "YOUR-APPLICATION-SESSION-KEY";

const unsigned long interval = 60000;

unsigned long previousMillis = 0;
unsigned int messageCounter = 0;

char message[50];
char receivedData[255];

byte receiveStatus = 0;
int channel = 0;

const sRFM_pins RFM_pins = {
  .CS = LORA_NSS,
  .RST = LORA_RST,
  .DIO0 = LORA_DIO0,
  .DIO1 = LORA_DIO1,
};

void setup() {
  Serial.begin(115200);
  delay(2000);

  pinMode(LORA_EN, OUTPUT);
  digitalWrite(LORA_EN, HIGH);
  delay(100);

  SPI.begin(SPI_SCK, SPI_MISO, SPI_MOSI, LORA_NSS);

  if (!lora.init()) {
    Serial.println("RFM95W radio was not detected");
    while (true) {
      delay(1000);
    }
  }

  Serial.println("RFM95W radio initialized");

  // A Class A device opens two receive windows after each uplink transmission.
  // Therefore, downlink messages are normally delivered after the device sends an uplink.
  lora.setDeviceClass(CLASS_A);

  // The data rate is configured using spreading factor 10 and a bandwidth of 125 kHz.
  // A higher spreading factor generally increases communication range and
  // receiver sensitivity but also increases airtime and reduces the data rate.
  lora.setDataRate(SF10BW125);

  // It will be available to multiple LoRaWAN channels.
```

```
  lora.setChannel(MULTI);

  // The transmission power should comply with the applicable regional frequency
  // regulations and the requirements of the LoRaWAN network.
  lora.setTxPower1(15);

  // ABP credentials.
  // Because ABP uses predefined session parameters,
  // the device does not perform an OTAA join procedure before transmitting data.
  lora.setNwkSKey(nwkSKey);
  lora.setAppSKey(appSKey);
  lora.setDevAddr(devAddr);

  Serial.println("LoRaWAN Class A ABP configuration completed");
}

void loop() {
  unsigned long currentMillis = millis();

  if (currentMillis - previousMillis >= interval) {
    previousMillis = currentMillis;

    snprintf(message, sizeof(message), "LoRa Counter-%u", messageCounter);

    Serial.print("Sending: ");
    Serial.println(message);

    lora.sendUplink(message, strlen(message), 0, 5);

    messageCounter++;

    channel = lora.getChannel();

    Serial.print("Channel: ");
    Serial.println(channel);
  }

  lora.update();

  receiveStatus = lora.readData(receivedData);

  if (receiveStatus > 0) {
    bool isText = true;

    for (int i = 0; i < receiveStatus; i++) {
      if (!(
        (receivedData[i] >= 32 && receivedData[i] <= 126) ||
        receivedData[i] == 10 ||
        receivedData[i] == 13
      )) {
        isText = false;
        break;
      }
    }

    if (isText) {
      Serial.print("Received text: ");

      for (int i = 0; i < receiveStatus; i++) {
        Serial.print(receivedData[i]);
      }
    } else {
      Serial.print("Received hexadecimal data: ");

      for (int i = 0; i < receiveStatus; i++) {
        if (static_cast<byte>(receivedData[i]) < 0x10) {
          Serial.print('0');
```

```
132           }
133
134           Serial.print(static_cast<byte>(receivedData[i]), HEX);
135           Serial.print(' ');
136         }
137       }
138
139       Serial.println();
140     }
141   }
```